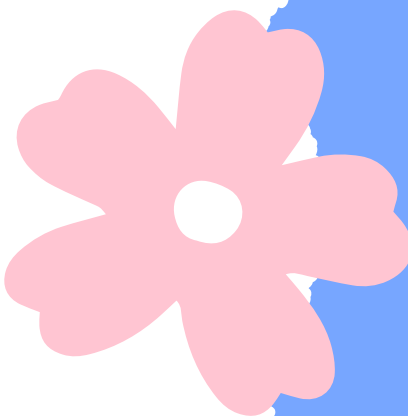
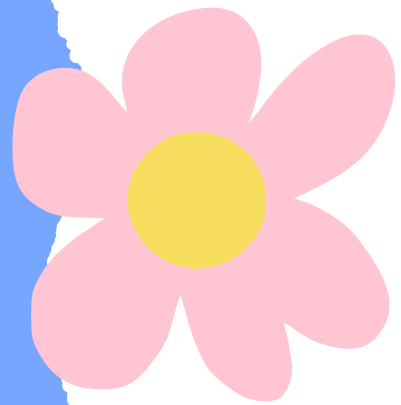
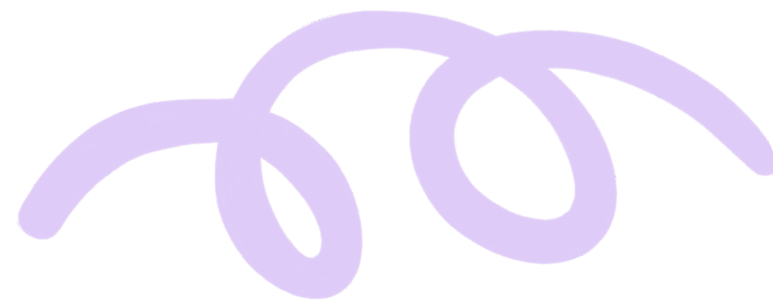




โครงการ Smart Expense Tracker by



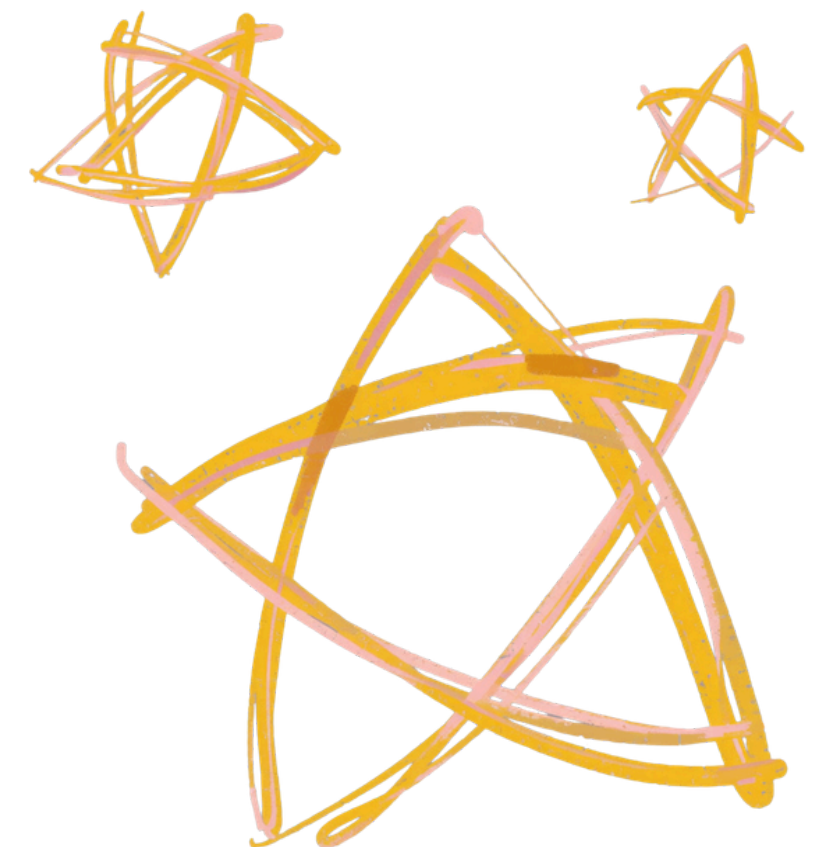
1. 6840205196 นางสาว ชัตติยา ปิตตะ
2. 6840205378 นางสาว ดาวิใจ บุญบุตร
3. 6840205287 ชนະชัย แส่นภูวา
4. 6840205477 ธรรมพร สงวนพฤกษ์





โจทย์คือ

หากเดือนนี้ผู้ใช้มีงบประมาณจำกัด
แต่มีรายการสิ่งที่อยากได้หรือ
ค่าใช้จ่ายผันแปรหลายรายการ
จะเลือกตัดหรือลดค่าใช้จ่าย
เพื่อให้ประหยัดเงินได้มากที่สุด





วัตถุประสงค์

- 
- 
1. เพื่อพัฒนาโปรแกรมบันทึกรายรับ-รายจ่าย ที่ใช้งานง่ายสะดวก และเหมาะสมกับบุคคลทั่วไป
 2. เพื่อให้ผู้ใช้งานสามารถมองเห็นสัดส่วนและพฤติกรรมค่าใช้จ่ายของตนเองผ่านระบบสรุปผลที่เป็นรูปธรรม



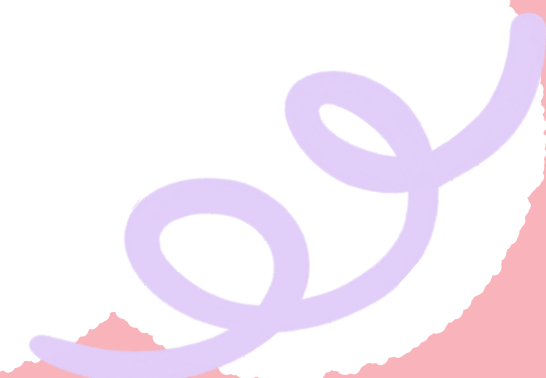
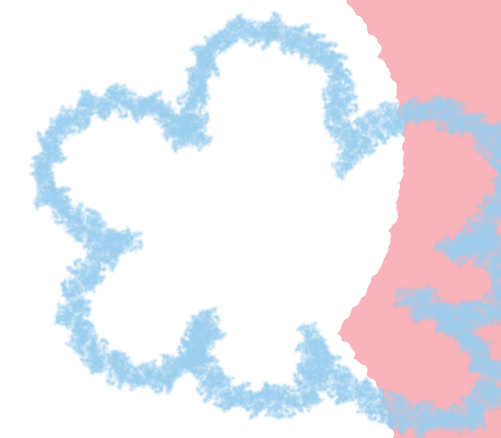


ประโยชน์ที่คาดว่าจะได้รับ



1. ผู้ใช้งานสามารถ**ควบคุม**และ**ลด**รายจ่ายที่ไม่จำเป็นลงจากการเห็นข้อมูลการเงินที่แท้จริง

2. ผู้ใช้งานมีวินัยทางการเงินมากขึ้นและสามารถวางแผนการออมเงินในอนาคตได้อย่างมีประสิทธิภาพ





โครงสร้างข้อมูลที่ใช้

1. Doubly Linked List

-เพื่อการท่องข้อมูลย้อนหลัง-เดินหน้าตามลำดับเวลา

2. Binary Search Tree / AVL Tree

-โดยใช้ "วันที่-เวลา" หรือ "จำนวนเงิน" เป็น Key
เพื่อให้ระบบสามารถค้นหาและเข้าถึงข้อมูล
แบบเป็นช่วงเวลา

ในการ Map หมวดห่มุค่าใช้จ่าย

3. Hash Table Chaining

4. Open Addressing

-เพื่อให้การจัดหมวดห่มุและสรุปลผลยอตรวม



ตัวอย่างโค้ด

บรรทัดที่ 1-8 (JSDoc Comment):

คอมเมนต์อธิบายชื่อไฟล์ และสรุปภาพรวมโครงสร้างข้อมูลทั้ง 3 ตัวที่นำมาใช้ร่วมกัน (DoublyLinkedList, AVLTree, SmartHashTable) รวมถึงอธิบายว่าระบบจะเซฟข้อมูลลง localStorage เสมอ

```
/**
 * store.js
 * ศูนย์กลางจัดการข้อมูลของแอป เชื่อมโยงโครงสร้างข้อมูลทั้ง 3 ชนิดเข้าด้วยกัน:
 * - DoublyLinkedList : เก็บลำดับธุรกรรมทั้งหมด (insert/delete O(1), เดินหน้า/ถอยหลัง)
 * - AVLTree : index ตาม timestamp สำหรับ Range Search แบบเร็ว
 * - SmartHashTable : สรุปยอดตามหมวดหมู่แบบเร็ว
 * ข้อมูลถูก persist ลง localStorage ทุกครั้งที่มีการเปลี่ยนแปลง เพื่อให้ Refresh แล้วข้อมูลไม่หาย
 */
```

บรรทัดที่ 9-11 (Key Constants):

กำหนดชื่อ Key ค่าคงที่ (STORAGE_KEY และ BUDGET_KEY) สำหรับอ้างอิงตำแหน่งในการอ่าน/เขียนข้อมูลลง localStorage

```
const STORAGE_KEY = 'smartbudget_transactions';
const BUDGET_KEY = 'smartbudget_budget';
```

บรรทัดที่ 12-19 (Class Constructor):

ประกาศคลาส Store และส่วนเริ่มต้น (Constructor) ทำหน้าที่สร้างอินสแตนซ์ของโครงสร้างข้อมูลทั้ง 3 ตัว แล้วเรียกเมธอด _load() เพื่อดึงข้อมูลมาเตรียมใช้งาน

```
class Store {
  constructor() {
    this.list = new DoublyLinkedList();
    this.tree = new AVLTree();
    this.hash = new SmartHashTable();
    this._load();
  }
}
```

บรรทัดที่ 20-32 (Loading Mechanism - _load):

อ่านข้อมูลจาก localStorage หากยังไม่มีข้อมูลเดิม ระบบจะสร้างข้อมูลตัวอย่าง (Sample Data) ตั้งต้นขึ้นมาก่อน จากนั้นส่งชุดข้อมูลไปแปลงโครงสร้าง

```
_load() {
  const raw = localStorage.getItem(STORAGE_KEY);
  let data;
  if (!raw) {
    data = generateSampleData();
    localStorage.setItem(STORAGE_KEY, JSON.stringify(data));
    localStorage.setItem(BUDGET_KEY, String(SAMPLE_BUDGET));
  } else {
    data = JSON.parse(raw);
  }
  this._rebuildStructures(data);
}
```

ตัวอย่างโค้ด

บรรทัดที่ 33-43 (Structure Rebuilding - `_rebuildStructures`): ล้างข้อมูลเก่าในโครงสร้างข้อมูลทั้ง 3 ออกทั้งหมด แล้วนำข้อมูล Array ล่าสุดมาหยอดลง DoublyLinkedList, AVLTree, และ SmartHashTable เพื่อให้ทุกโครงสร้างซิงค์ข้อมูลตรงกัน

```
_rebuildStructures(data) {  
  this.list.clear();  
  this.tree = new AVLTree();  
  this.hash.clear();  
  data.forEach(t => {  
    this.list.append(t);  
    this.tree.insert(new Date(`${t.date}T${t.time ||  
    '00:00'}:00`).getTime(), t.id);  
    if (t.type === 'expense') this.hash.add(t.category, t.amount);  
  });  
}
```

บรรทัดที่ 44-48 (Persistence - `_persist`): ดึงข้อมูลปัจจุบันจาก DoublyLinkedList แปลงให้อยู่ในรูปแบบข้อความ JSON แล้วบันทึกลง localStorage เพื่อป้องกันข้อมูลหายเมื่อ Refresh หน้าเว็บ

```
_persist() {  
  const data = this.list.toArray(true).map(n => n.toJSON());  
  localStorage.setItem(STORAGE_KEY, JSON.stringify(data));  
}
```

ตัวอย่างโค้ด

บรรทัดที่ 49-56 (Budget Management - `getBudget` / `setBudget`): ฟังก์ชันสำหรับอ่านค่า (Get) และบันทึกค่า (Set) งบประมาณทางการเงินจาก `localStorage`

```
getBudget() {  
  return Number(localStorage.getItem(BUDGET_KEY)) || 0;  
}  
  
setBudget(amount) {  
  localStorage.setItem(BUDGET_KEY, String(amount));  
}
```

บรรทัดที่ 57-60 (ID Generator - `genId`): สร้างรหัสประจำตัวรายการ (Unique ID) แบบสุ่ม โดยผสมเวลาปัจจุบันกับตัวเลขสุ่ม เพื่อใช้เป็นคีย์อ้างอิงธุรกรรม

```
genId() {  
  return 'tx' + Date.now().toString(36) +  
    Math.floor(Math.random() * 1000);  
}
```

ตัวอย่างโค้ด

บรรทัดที่ 61-69 (Add Transaction - addTransaction):
รับข้อมูลธุรกรรมใหม่ ใส่ ID แล้วบันทึกเพิ่มลงในโครงสร้าง
ข้อมูลทั้ง 3 ชนิดพร้อมกัน ก่อนสั่งเซฟลง localStorage

```
addTransaction(data) {  
  const record = { ...data, id: this.genId() };  
  this.list.append(record);  
  this.tree.insert(new Date(`${record.date}T${record.time ||  
'00:00'}:00`).getTime(), record.id);  
  if (record.type === 'expense') this.hash.add(record.category,  
record.amount);  
  this._persist();  
  return record;  
}
```

บรรทัดที่ 70-90 (Update Transaction -
updateTransaction): แก้ไขข้อมูลธุรกรรมที่มีอยู่ โดยลบยอด
เงิน/Timestamp เก่าออกจาก SmartHashTable และ AVLTree แล้ว
อัปเดตค่าใหม่เข้าไปแทน รวมถึงปรับข้อมูลใน DoublyLinkedList

```
updateTransaction(id, newData) {  
  const node = this.list.find(id);  
  if (!node) return false;  
  // ปรับ hash table: หักยอดเก่าออก แล้วบวกยอดใหม่เข้า (ถ้าเป็นรายจ่าย)  
  if (node.type === 'expense') this.hash.remove(node.category, node.amount);  
  if (newData.type === 'expense') this.hash.add(newData.category,  
newData.amount);  
  // ปรับ AVL tree ถ้าวันเวลาเปลี่ยน  
  const oldTs = node.timestamp;  
  const newTs = new Date(`${newData.date}T${newData.time ||  
'00:00'}:00').getTime();  
  if (oldTs !== newTs) {  
    this.tree.remove(oldTs, id);  
    this.tree.insert(newTs, id);  
  }  
  this.list.update(id, newData);  
  this._persist();  
  return true;  
}
```

ตัวอย่างโค้ด

บรรทัดที่ 91-100 (Delete Transaction - deleteTransaction): ค้นหารายการจาก ID และทำการลบข้อมูลนั้นออกจากทั้ง 3 โครงสร้างข้อมูล แล้วบันทึกผลลัพธ์ลง localStorage

```
deleteTransaction(id) {  
  const node = this.list.find(id);  
  if (!node) return false;  
  if (node.type === 'expense') this.hash.remove(node.category,  
node.amount);  
  this.tree.remove(node.timestamp, id);  
  this.list.remove(id);  
  this._persist();  
  return true;  
}
```

บรรทัดที่ 101-104 (Get All Data - getAll): ดึงรายการธุรกรรมทั้งหมดจาก DoublyLinkedList ออกมาในรูปแบบ Array (เลือกเรียงจากใหม่ไปเก่า หรือเก่าไปใหม่ได้)

```
getAll(forward = false) {  
  return this.list.toArray(forward);  
}
```

บรรทัดที่ 105-112 (Date Range Search - searchByDateRange): ค้นหาธุรกรรมตามช่วงเวลา โดยใช้ AVLTree ค้นหาช่วง Timestamp อย่างรวดเร็ว เพื่อเอา ID ไปดึงข้อมูลฉบับเต็มจาก DoublyLinkedList

```
// ค้นหาด้วยช่วงเวลาโดยใช้ AVL Tree range search แล้วดึง  
รายละเอียดจาก linked list index  
searchByDateRange(startDate, endDate) {  
  const startTs = startDate ? new  
Date(`${startDate}T00:00:00`).getTime() : -Infinity;  
  const endTs = endDate ? new  
Date(`${endDate}T23:59:59`).getTime() : Infinity;  
  const ids = this.tree.rangeSearch(startTs, endTs);  
  return ids.map(id => this.list.find(id)).filter(Boolean);  
}
```

บรรทัดที่ 113-116 (Category Totals - getCategoryTotals): ดึงสรุปยอดรวมรายจ่ายจำแนกตามหมวดหมู่โดยตรงจาก SmartHashTable

```
getCategoryTotals() {  
  return this.hash.entries();  
}
```

ตัวอย่างโค้ด

บรรทัดที่ 117-129 (Financial Summary - summary): คำนวณและคืนค่าสรุปภาพรวมการเงินของระบบ เช่น ยอดรายรับรวม, ยอดรายจ่ายรวม, เงินคงเหลือ, งบประมาณ และจำนวนรายการทั้งหมด

```
summary() {  
  const all = this.getAll(true);  
  const income = all.filter(t => t.type === 'income').reduce((s, t) => s +  
t.amount, 0);  
  const expense = all.filter(t => t.type === 'expense').reduce((s, t) => s +  
t.amount, 0);  
  return {  
    totalIncome: income,  
    totalExpense: expense,  
    balance: income - expense,  
    budget: this.getBudget(),  
    count: all.length  
  };  
}
```

บรรทัดที่ 130-137 (Reset Data - resetToSample): ล้างข้อมูลเดิมใน localStorage ทั้งหมดออก แล้วทำการโหลดข้อมูลตัวอย่างเริ่มต้นกลับมาใช้อีกครั้ง

```
resetToSample() {  
  localStorage.removeItem(STORAGE_KEY);  
  localStorage.removeItem(BUDGET_KEY);  
  this._load();  
  this._persist();  
}
```

Scope

- พัฒนาระบบจัดเรียงข้อมูลค่าใช้จ่ายตามเงื่อนไขต่างๆ
(เช่น ยอดเงิน, หมวดหมู่, วันที่) โดยประยุกต์ใช้และ
เปรียบเทียบประสิทธิภาพของอัลกอริทึมการจัดเรียง

เช่น Quick Sort และ Merge Sort

- มีระบบสืบค้นข้อมูลธุรกรรมย้อนหลังแบบระบุช่วงเวลา
(Range Search) และคีย์เวิร์ด โดยใช้ประโยชน์จากโครงสร้าง
ข้อมูลที่เหมาะสม

- พัฒนาฟังก์ชันแนะนำการปรับลดงบประมาณ
(Budget Optimization) เมื่อมีการแจ้งเตือน
ว่าเงินใกล้เกินงบ โดยประยุกต์ใช้

Greedy Algorithm หรือ Knapsack Algorithm